



HOPPER INFORMATION SERIES

04 / TEXT CODING

JavaScript API reference

The stable variables and functions available to advanced Hopper Studio programs.

ASYNC + AWAIT

DRONE

VISION

RUNTIME





Your program receives four useful bindings

D drone

Flight, state and accessory commands shared by the real and simulated aircraft.

V vision

Thresholding, object detection, custom classification and AprilTag functions.

R runtime

Keys, drone events, loops, cancellation and cooperative yielding.

C console

log, warn and error lines in the in-app program console.

TOP-LEVEL `await` WORKS

```
await drone.takeOff();
```

Promise-returning calls must be awaited or the main body may finish and auto-land early.



drone: core flight commands

```
await drone.takeOff();
```

No args. Promise<void>; await. Rejects on low battery (<=10%) or BLE failure; a stopped run returns early.

```
await drone.land();
```

No args. Promise<void>; await. Zeros axes, sends land, then waits about 5 s; transport failures reject.

```
await drone.hover();
```

No args. Promise<void>; await. Zeros every axis, waits about 1 s, and returns void.

```
await drone.fly("forward", 1, 15);
```

direction: up/down/left/right/forward/backward; seconds >=0 (default 0); power -100..100 (default 0). Promise<void>; await.

```
await drone.rotate(90, "clockwise");
```

degrees >=0 (default 0); direction clockwise/counterclockwise (default clockwise). Promise<void>; await; open-loop timing.

```
await drone.flip("forward");
```

direction: forward/backward/left/right. Promise<void>; await. Missing acknowledgement rejects; preserve safe clearance.

```
drone.setAxis("pitch", 15);
```

axis: pitch/roll/yaw/gaz/altitude; power clamps -100..100. Synchronous void; persists until reset; altitude aliases gaz.

```
drone.reset();
```

No args. Synchronous void. Zeros all persistent motion axes; it does not land.

All Promise-returning calls must be awaited. Stop cancels the active run; UI STOP & LAND also sends a landing command.



drone: state, timing and accessories

```
await drone.wait(1);
```

seconds: number ≥ 0 . Promise<void>; await. Resolves after the delay or early when the run stops.

```
const level = drone.getBatteryLevel();
```

No args. Returns number 0..100 or null before telemetry; synchronous and does not throw.

```
const flying = drone.isFlying();
```

No args. Returns boolean from the controller's flying/landed state; synchronous.

```
const landed = drone.isLanded();
```

No args. Returns boolean from the controller's flying/landed state; synchronous.

```
await drone.waitUntilBatteryLevelChanges();
```

No args. Promise<void>; await. Resolves on a new battery value or when the run stops.

```
await drone.takePicture();
```

No args. Promise<void>; await. Requests a camera photo; camera or transport errors reject.

```
await drone.grabber("OPEN");
```

action: "OPEN" or "CLOSE". Promise<void>; await. Rejects when no physical grabber is attached; simulator is a timing stub.

```
await drone.fireGun();
```

No args. Promise<void>; await. Rejects when no physical cannon is attached; simulator is a timing stub.

```
await drone.cutoff();
```

No args. Promise<void>; await. Immediate motor cutoff for emergencies; transport errors reject.

APP-OWNED - DO NOT USE IN STUDENT PROGRAMS

disconnect, abortRun, startRun, stopRun, landNoWait, forceLand, manualNudge, onTelemetry and onEvent are lifecycle/UI internals. Replacing callbacks can break the app.



vision: binary and COCO functions

```
const mask = await vision.scanThreshold(60, false);
```

threshold: 0..100% (default 60); invert: boolean (default false). Promise<ThresholdResult>; fresh frame; camera failure rejects.

```
const seen = await vision.seesBinary("white", 60, false, 10);
```

color: white/black; threshold + minimumCoverage: 0..100%; invert boolean. Promise<boolean>; defaults 60/false/10; camera failure rejects.

```
const centered = await vision.binaryCenter("white", 60, false);
```

color: white/black; threshold 0..100% default 60; invert default false. Promise<boolean>; tests center pixel of a fresh frame.

```
await vision.loadObjectModel();
```

No args. Promise<ObjectDetection>; await. Lazy-loads local COCO-SSD; file://, model-load, or network/server failures reject.

```
const objects = await vision.detectObjects(0.55);
```

minimumConfidence: 0..1 (default .55). Promise<VisionDetection[]>; up to 10 results. Camera/model failures reject.

```
const apple = await vision.seesObject("apple", 0.55);
```

label: string; minimumConfidence: 0..1 default .55. Promise<boolean>; performs a fresh scan; camera/model failures reject.

```
const x = vision.objectCoordinate("apple", "x", 0.55);
```

label string; axis x/y; confidence 0..1 default .55. Synchronous number -100..100 from saved detections; returns 0 if absent.

ThresholdResult: threshold, invert, whiteCoverage, blackCoverage, centerWhite, frameWidth, frameHeight, binaryData

VisionDetection adds class, score, bbox and centerX/centerY; right and up are positive.



vision: AprilTags and custom models

```
const tags = await vision.scanAprilTags();
```

No args. Promise<AprilTagDetection[]>; await. Captures a fresh frame; a missing camera/canvas rejects.

```
const found = await vision.seesAprilTag("any");
```

id: "any" (default) or tag36h11 ID 0..586. Promise<boolean>; await; scans first; camera failures reject.

```
const aligned = await vision.centerOnAprilTag(drone, "any", 10, 5, 5, 3);
```

args: drone; id; power 0..100; centerSlack 1..35; angleSlack 1..45; lostSearches 1..20. Promise<boolean>; 30 s limit.

```
const classes = await vision.classifyCustomModel();
```

No args. Promise<CustomPrediction[]>; await. Requires UI-loaded model + camera; otherwise rejects.

```
const pad = await vision.seesCustomLabel("landing pad", 0.75);
```

label: string; minimumConfidence: 0..1 (default .75). Promise<boolean>; await; scans whole frame; model/camera failures reject.

APRILTAG RETURN

Detections include ID, x/y center in -100..100, image corners and 2D image yaw. Centering returns false on loss, stop, or timeout.

CUSTOM MODEL FILES

Load model.json, weights.bin and metadata.json in Vision Testing. Predictions are className + probability; there are no boxes.



runtime and console

```
runtime.registerKey("pressed", "a", async () => {});
```

kind: pressed/released; key string; async handler. Returns void; handlers may overlap; rejection is reported to the console.

```
runtime.registerDrone("landed", async () => {});
```

event: flying/landed/crashed/batteryLevelChanged; async handler. Returns void; handlers may overlap; rejection is reported.

```
await runtime.runBlock("block-id", async () => {});
```

blockId: string; async/value handler. Promise<handler result>; await. A stopped runtime rejects; active-block state clears in finally.

```
const down = runtime.keyIsPressed("ArrowUp");
```

key: string. Returns live boolean synchronously; no await and no expected failure.

```
await runtime.repeatForSeconds(5, async () => {});
```

seconds >=0; async handler. Promise<void>; await. Iterations are sequential and yield; stopping throws Program stopped.

```
await runtime.tick();
```

No args. Promise<void>; await. Yields one event-loop turn; rejects with Program stopped after cancellation.

```
const { stopped, hasEvents } = runtime;
```

Both are booleans. stopped reports cancellation; hasEvents becomes true after event registration. Synchronous reads.

```
runtime.stop();
```

No args. Synchronous void. Unregisters and aborts runtime tasks; idempotent; it does not command a landing.

```
console.log("x =", x);
```

One or more values. log/warn/error return void and append a line to the app console; no await.

Use the red UI STOP & LAND for operator intervention: runtime.stop() alone does not land.



A complete safe example

WHITE-PAPER-LANDING.JS

```
1  await drone.takeOff();
2  try {
3    for (let attempt = 0; attempt < 20; attempt += 1) {
4      const overPaper = await vision.binaryCenter("white", 60, false);
5      if (overPaper) break;
6      await drone.fly("forward", 0.5, 12);
7    }
8  } finally {
9    await drone.land();
10 }
```

WHY THIS PATTERN

- Top-level await keeps order.
- Fresh vision decision after each move.
- Twenty attempts bound the search.
- Small, conservative command.
- finally sends land after error or break.

REMEMBER

- No GPS or altitude variable.
- Vision can be wrong - test lighting first.
- UI Stop & Land remains the operator control.
- Keep the flight area clear.



Use the documented surface

UNITS AT A GLANCE

- Flight power: percent, recommend 0-100.
- Fly/wait time: seconds.
- Rotation: degrees + direction.
- JS confidence: fraction 0-1.
- Block confidence/threshold/coverage: percent 0-100.
- Vision x/y: -100..100.

SOURCES AND FURTHER READING

Shared drone controller contract

lib/drone.ts in the Hopper Studio project

Vision runtime

lib/vision.ts and lib/apriltags.ts in the Hopper Studio project

Program runtime and app execution

lib/runtime.ts and components/HopperStudio.tsx

AWAIT FLIGHT + VISION CALLS

Use documented members. Treat browser globals and app internals as implementation details.